Software Engineering in Machine Learning for Vehicle Routing

Final Assignment: WASP Software Engineering and Cloud Computing

Spring 2026

Filip Rydin - Electrical Engineering, Chalmers

1 Introduction

In this report, I detail some of the core principles discussed in the software module of the course, with a particular focus on how they apply to my research area: machine learning for combinatorial optimization problems.

Specifically, in my area, we investigate the use of Reinforcement Learning (RL) to learn how to solve graph-based optimization problem. In particular, my focus is on Vehicle Routing Problems (VRPs), which typically surface in logistics applications. VRPs can be thought of as extensions to the Traveling Salesperson Problem (TSP). Since this class of problems is NP-hard, large problem instances are typically infeasible to solve in real-time, motivating the use of RL-based methods as fast surrogates.

2 Lecture Principles

2.1 Principle 1: Automated Testing/QA

After this discussion during class, I think I appreciate the value of these parts more. Specifically, when it comes to my research a very simple check I can do is whether the algorithm outputs a valid solution or not. I have implemented some of these tests before, but should really be more thorough with this. Especially considering such tests are normally easy to run, since verifying whether a solution is feasible or not is computationally tractable. Conducting such tests will not guarantee that the code is fail safe, but it will prevent issues where I think the model outperforms the baselines, but it actually is outputting infeasible solutions. Such errors can be quite dangerous unless addressed.

2.2 Principle 2: Automation via Coding Agents

One of the main takeaways for me, personally, from this course is that I will consider automating more of my workflows. In the discussions during class, I feel that it was highlighted just how useful coding agents have become. For example, I had a person in my discussion group who used Codex in a brilliant way to help with finding related work. Previously, I have made used of AI mainly via the online chat applications as

well as copilot through Visual Studies code, but I will now try Codex and Claude Code, to see if I can find further use for these in my research.

3 Guest-Lecture Principles

3.1 Principle 1: Requirements Engineering (guest lecture 1)

Regarding the first guest lecture, treating requirements engineering, I think I was previously quite unfamiliar with this concept. I can certainly see the appeal of a structured first-stage in software design, where stakeholders are elicited, goals are modeled and the software is designed, before any code is written. I am contemplating whether I should implement some similar structure in my research process. Specifically, for my topic, when I plan a new repository or project, I should probably start by thinking about: 1) What routing problems in particular do I want to solve; 2) What baselines do I want to compare against; and 3) what metrics do I want to log. These questions can then influence the design of the system.

3.2 Principle 2: Behavioral Engineering (guest lecture 2)

Specifically, I think the discussion around degree of challenge vs motivation in this guest lecture was very interesting. The "hill" shaped plot showing how moderately challenging task are the most motivating, and the accompanying bar-chart showing where most work tends to lie, was quite interesting to see. To an extent it is obvious, but it is also not something I have considered before. I think this "analysis-framework" will help my research process (not necessarily the research topic itself). For instance, while I have previously planned my days to take into account task intensiveness, I think I will do so more explicitly now. I also agree with the sentiment during the lecture discussion, that LLMs and multi-agent systems unfortunately shift the work distribution to higher intensity tasks, which causes fatigue in the long run. This can be worth it to keep in mind in the future.

4 Data Scientists, Software Engineers, and AI Engineers

4.1 Differences between data scientists and software engineers according to the book

According to the book "Machine Learning in Production" [1], data scientists adopt a model-centric workflow. At its core, this involves improving and validating models in an almost research-like setting.

On the other hand, software engineers handle the wider system, including non-ML components and components reacting with the ML-parts. As the name implies, they are more "engineers" rather than scientists.

4.2 My thoughts on this distinction

Generally I do think I agree. These roles have somewhat different scopes and probably require different skill-sets. It is likely warranted to let these roles be kept by different people. That said, I would agree with the paper by Sculley et. al. [2], which argues that integrating these roles into mixed teams might be effective. This ensures a more holistic approach to model development and deployment.

4.3 The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area?

It certainly applies to a great extent. If I were to apply an ML-based VRP solver in real life, it would be important to think about robustness, safety and human interaction. Is the vehicle going to be completely autonomous, then we can probably not allow badly planned routes, the system needs to be more robust than if there is a human in the loop. Also, if everyone uses these solvers, it could create a negative feedback loop, where some roads become overcrowded.

Moreover, the system-level requirements also influence the model requirements, as discussed in the book. Is latency a major requirement? Then maybe ML-based systems are more appropriate. Or is exact route optimality preferable? Then exact solvers, perhaps augmented with ML-components, are more suitable.

4.4 Will data scientists and software engineers remain distinct roles?

As previously argued, I think keeping the roles somewhat separate is a sound idea, but I also think everyone should have general knowledge about other parts of the organization and software stack.

4.5 Where does "AI Engineering" fit in this picture?

To an extent I think it is almost a buzzword for software engineer for AI systems, but I can also see the perspective that it is something more distinct, modern. Maybe it takes more into account the modern AI/ML/LLM stack, including prompting, multi-agent systems and RAG.

4.6 How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

They certainly make them more blurry. If someone can "vibe" together an entire software stack for an ML-application, maybe there is no more need for software engineers, and data scientists can fill both roles. Conversely, if software engineering turns out to be the difficult part, and the research level is more suitable for agents, maybe a software engineer can cover both roles. Or perhaps AI-engineers will fill both roles, using multi-agent systems where appropriate to automate system building.

5 Paper Analysis

5.1 Paper 1: Rule-Based Assessment of Reinforcement Learning Practices Using Large Language Models (CAIN 2025) [3]

5.1.1 Core ideas and their SE importance

The paper shows that LLMs can assist in creating best-practice reinforcement learning pipelines/environments, by checking for rule violations. Examples of such rules, in general, not necessarily discussed in the paper, include: ensuring that trained models are versioned and stored in a model registry; separating training and evaluation pipelines so that performance is measured on held-out data rather than on the training environment; using a standardized RL framework (e.g., RLlib or Stable-Baselines) rather than ad-hoc implementations; logging hyperparameter configurations alongside training runs to support reproducibility; and enforcing proper coordination patterns when deploying multi-agent systems (e.g., avoiding shared mutable state between agents).

In the paper, they show that LLMs achieve better performance than heuristics on identifying such best-practices. From an engineering perspective, such checks can be valuable to ensure systems are scalable, robust and transparent.

5.1.2 Relation to my research

I use Reinforcement Learning to train my models, hence I should ideally follow the best-practices that the rules in the paper cover. In a production-system, actually verifying that these rules are followed could be critical to minimize technical debt.

5.1.3 Integration into a larger AI-intensive project

The paper actually includes a case-study. In this case-study the authors integrate their approach into the MLOps pipeline of a company, to assist with code diagnostics.

From a broader perspective, if a reinforcement learning-based model is at the core of a software stack, the ideas in this paper could help to verify that the training and deployment of this model is done correctly. By using the LLM assistance, a team can offload the otherwise tedious task of manually auditing RL code to an automated service. The LLM-based detectors are particularly valuable for rules that are hard to capture with classic heuristics, because they understand the intent of the code rather than just its structure. This means the rule set can be extended with new best practices simply by writing natural-language rule descriptions, without needing to write new custom heuristics for every rule.

For instance, considering my own research, if a VRP solver trained with RL is to be deployed in a wider system, it would be massively helpful to know that the training and evaluation pipeline follows the industry-practice. Checking this manually is possible, but automated checks are possibly more reliable and minimize the risk of human errors.

5.1.4 How my research could be adapted

For one, I could directly implement some of the rules from the paper that I do not follow. For instance, I do not have an automated pipeline for hyperparameter tuning.

But I could also apply LLMs (for instance via Claude code or Codex) to systematically evaluate my code. I think that just asking the LLM to check whether the code base adheres to best-practices would go a long way.

5.2 Paper 2: Investigating Issues that Lead to Code Technical Debt in Machine Learning Systems (CAIN 2025) [4]

5.2.1 Core ideas and their SE importance

The paper investigates reasons behind technical debt. That is, they do not categorize and identify the debt, they instead focus on decisions, errors and processes that increase the risk of accruing the debt in the first place. The authors first identify some candidate risk and then conduct interviews with practitioners to evaluate the prominence of these risk. To take an example, the authors identify wrong/misaligned evaluation as a significant risk. If such misalignment occurs, it can lead to wrong models or settings being deployed in production, which can lead to robustness issues and possibly the need to spend much time fixing issues later. Another example is if there are features that are missing occasionally. One practitioner highlights that this can lead to engineers training two models, one without the feature and one with it, which will lead to increased maintenance cost. Engineers of AI systems should take into account such risk and address them, to reduce the drivers behind technical debt.

5.2.2 Relation to my research

The paper is highly relevant to my research. By understanding the factors that *complicate* my model pipelines, I can design smart and flexible solutions to avoid incurring technical debt. On the other hand, if I avoid addressing these factors directly, it can lead to ad-hoc solutions that will be difficult to deal with later. One example I often encounter is training/testing across different environments. Different environments can have different features, which must be handled smartly through a flexible pipeline. If I instead hard-code solutions into the model itself, which I unfortunately have done, it will lead to head-aches later on.

5.2.3 Integration into a larger AI-intensive project

Consider a real-world route planning system, with perhaps an ML-based model as a component. It is highly likely that many of the risk factors from the paper exist in this system. Considering the four categories of risk the authors propose in the paper:

- **Data collection:** In this stage, it is highly likely that many of the data sources the routing rely on (e.g., traffic measurements, weather) are unreliable and error-prone. This can lead to engineers designing complicated ad-hoc solutions to handle special cases.

- **Data pre-processing:** Similarly, in this stage, when the raw data is processed, engineers must ensure robust and scalable pipelines. If scaling, re-balancing, feature extraction or similar are very complicated, or case-dependent, there is significant risk of technical debt being introduced into the system.
- **Model creation and training:** In this stage, engineers and data scientists must ensure training, hyperparameter tuning and model selection is performed in ways that mitigate risks of future technical debt. For example, by trading off model complexity versus accuracy in a reasonable way, much technical debt may be avoided.
- **Model evaluation** Lastly, in this stage, as discussed above, robust evaluation and metric selection is important to ensure the model actually works in real-life.

In all these stages, by identifying the risk-factors associated with technical debts, engineers may create systems and pipelines that are more robust and scalable.

5.2.4 How my research could be adapted

When I code my pipelines, I tend to solve problems as a encounter them somewhat ad-hoc. I think, following this paper and the discussions during class, that I will be somewhat more deliberate in identifying particular risks. Then, I can plan at least slightly more deliberately, which might avoid much technical debt later on. Hopefully this will lead to fewer special-cases needing handling.

6 Evidence, Originality, and GenAI-Use Transparency

References:

1. C. Kästner. Machine Learning in Production: From Models to Products. MIT Press. 2025.
2. D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in Machine learning systems. In Proceedings of the 29th International Conference on Neural Information Processing Systems (Neurips). 2015
3. E. Ntentos, S. J. Warnett, and U. Zdun. Rule-Based Assessment of Reinforcement Learning Practices Using Large Language Models. In IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN). 2025
4. R. Ximenes, A. P. S. Alves, T. Escovedo, R. Spinola, and M. Kalinowski. Investigating Issues That Lead to Code Technical Debt in Machine Learning Systems. In IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN). 2025.

Source analysis: [2, 3, 4] are conference papers, while [1] is a book. Hence, I think they are all quite trustworthy, having undergone some form of peer-review.

GenAI use: I used GenAI to brainstorm for Sections 4 and 5 as well as to identify papers for Section 5. All text is written without the use of generative AI.